

UNIVERSITAS GADJAH MADA

Departemen Ilmu Komputer dan Elektronika

Tutorial OpenMP

Pemrograman Paralel CPU dengan OpenMP dalam C: Delapan Exercise dan Mini-Project
Perkalian Matriks

Nama: Aufa Akmal Bunaya

NIM: 23/515767/PA/22027

Repository GitHub: github.com/aufaakmalbunaya/openmp-tutorial

4 Oktober 2026

Daftar Isi

1	Pendahuluan	2
1.1	Lingkungan dan persiapan	2
2	Exercise 1: Membuat tim thread	2
3	Exercise 2: Data shared dan private	3
4	Exercise 3: Paralelisasi penjumlahan vektor	4
5	Exercise 4: Akumulasi aman dengan reduction	4
6	Exercise 5: Perbandingan loop schedule	5
7	Exercise 6: Koordinasi fase dependen	6
8	Exercise 7: Membuat task independen	7
9	Exercise 8: Kebenaran numerik dan speedup	7
10	Mini-project: Perkalian matriks paralel	9
11	Refleksi: satu bug dan diagnosisnya	10
12	Reproduksi	10
13	Kesimpulan	11

Pendahuluan

Tugas ini mengerjakan tutorial pemrograman paralel OpenMP dalam C pada satu virtual machine: delapan exercise berurutan yang mencakup model eksekusi thread, atribut data-sharing, paralelisasi loop, reduction, penjadwalan loop, sinkronisasi fase dependen, dan task, ditutup dengan benchmark numerik π untuk mengukur speedup serta sebuah mini-project perkalian matriks paralel. Setiap exercise mencakup program lengkap, perintah eksekusi, perilaku yang diharapkan, dan pertanyaan investigasi; seluruhnya dijalankan secara nyata dan jawabannya didasarkan pada perilaku yang teramati.

Lingkungan dan persiapan

Seluruh pekerjaan dijalankan pada VM dengan spesifikasi pada Tabel 1 dan toolchain pada Tabel 2. Jumlah thread yang diuji adalah 1, 2, dan 4; karena VM hanya memiliki 2 vCPU, konfigurasi 4 thread merupakan oversubscription yang disengaja untuk diamati perilakunya.

Tabel 1: Spesifikasi virtual machine.

Komponen	Nilai
Hostname	htch-runtime
Sistem operasi	Ubuntu 24.04.5 LTS
CPU	2 vCPU AMD EPYC 9D25
Memori	7.7 GiB

Tabel 2: Toolchain kompilasi.

Komponen	Nilai
GCC	13.3.0
Flag kompilasi	-std=c11 -O2 -Wall -Wextra -fopenmp
Flag link tambahan	-lm untuk program yang memakai math.h
Variabel lingkungan	OMP_DYNAMIC=FALSE, thread via OMP_NUM_THREADS

```
student@htch-runtime:~/openmp-lab$ gcc --version | head -1; nproc; grep -m1 'model name' /proc/cpuinfo
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
2
model name      : AMD EPYC 9D25 126-Core Processor
student@htch-runtime:~/openmp-lab$ make clean >/dev/null && make 2>&1 | tail -3
gcc -std=c11 -O2 -Wall -Wextra -fopenmp src/tasks.c -o bin/tasks -fopenmp
gcc -std=c11 -O2 -Wall -Wextra -fopenmp src/pi_benchmark.c -o bin/pi_benchmark -fopenmp -lm
gcc -std=c11 -O2 -Wall -Wextra -fopenmp src/matmul.c -o bin/matmul -fopenmp -lm
student@htch-runtime:~/openmp-lab$
```

Gambar 1: Pemeriksaan lingkungan dan kompilasi ulang seluruh program dengan `make`.

Keempat belas program (8 exercise, 5 modifikasi, 1 mini-project) dibangun dengan satu perintah `make` memakai Makefile pada bagian Reproduksi. Tidak ada warning kompilasi.

Exercise 1: Membuat tim thread

Berkas: `src/hello_openmp.c`. Program mencetak ukuran tim sebelum dan sesudah region paralel; setiap thread mencetak sapaan di dalam `#pragma omp critical`.

```

student@htch-runtime:~/openmp-lab$ OMP_DYNAMIC=FALSE OMP_NUM_THREADS=4 ./bin/hello_omp
Before: team size = 1
Hello from thread 3 of 4
Hello from thread 1 of 4
Hello from thread 2 of 4
Hello from thread 0 of 4
After: team size = 1
student@htch-runtime:~/openmp-lab$ OMP_DYNAMIC=FALSE OMP_NUM_THREADS=4 ./bin/hello_omp
Before: team size = 1
Hello from thread 3 of 4
Hello from thread 2 of 4
Hello from thread 1 of 4
Hello from thread 0 of 4
After: team size = 1
student@htch-runtime:~/openmp-lab$ █

```

Gambar 2: Dua eksekusi dengan 4 thread: urutan sapaan berbeda antar run, tetapi setiap thread tepat mencetak satu baris.

Hasil investigasi:

1. Thread yang diminta vs aktual: `OMP_NUM_THREADS=1, 2, 4` semuanya dipenuhi persis.
2. Urutan sapaan berubah antar run karena penjadwalan thread oleh OS tidak deterministik; `critical` hanya menyekur akses, tidak mengurutkan ID thread.
3. `printf` sebelum region dieksekusi tepat satu kali (oleh thread awal), sedangkan di dalam region dieksekusi sekali per thread — region paralel dieksekusi penuh oleh setiap anggota tim.
4. `After` selalu muncul terakhir karena akhir region paralel memiliki barrier implisit: thread awal baru melanjutkan setelah seluruh tim tiba dan tersinkronisasi.

Exercise 2: Data shared dan private

Berkas: `src/data_sharing.c`, memakai `#pragma omp parallel default(none) firstprivate(base) private(scratch)`.

```

student@htch-runtime:~/openmp-lab$ OMP_NUM_THREADS=4 ./bin/data_sharing
thread=3 base=13 scratch=9
thread=1 base=11 scratch=1
thread=2 base=12 scratch=4
thread=0 base=10 scratch=0
Outside: base=10 scratch=-1
student@htch-runtime:~/openmp-lab$ █

```

Gambar 3: Setiap thread mendapat salinan privat `base=10+t` dan `scratch=t2`; variabel asli di luar region tidak berubah.

Hasil investigasi:

1. Perubahan `base` di dalam region tidak memengaruhi `base` asli (`Outside: base=10`) karena `firstprivate` memberi tiap thread *salinan privat* yang diinisialisasi dengan nilai 10.
2. Mengganti `private(scratch)` menjadi `firstprivate(scratch)` lalu mencetak nilainya sebelum assignment: setiap thread mengamati nilai awal `-1` (salinan dari nilai asli). Ini membuktikan `firstprivate` menyalin nilai masuk, sedangkan `private` tidak.

3. Membaca `private(scratch)` sebelum assignment pertama tidak valid: variabel privat tidak diinisialisasi, nilainya tak terdefinisi — nilai “kebetulan” yang tercetak tidak boleh dianggap bermakna.
4. Menghapus `firstprivate(base)` dengan `default(none)` membuat kompilasi gagal: `error: 'base' not specified in enclosing 'parallel'`. Klausula `default(none)` memaksa setiap variabel yang dipakai di dalam region dideklarasikan atribut data-sharing-nya secara eksplisit.

Catatan: `firstprivate` pada sebuah *pointer* hanya menyalin alamatnya; seluruh thread tetap menunjuk memori yang sama. Kepemilikan memori harus diputuskan terpisah dari kepemilikan variabel pointer.

Exercise 3: Paralelisasi penjumlahan vektor

Berkas: `src/vector_add.c` ($n = 10^6$), memakai `#pragma omp parallel for default(none) shared(a,b,c,n) schedule(static)`. Penyimpanan array bersifat shared; variabel loop `i` privat menurut aturan worksharing-loop OpenMP.

```
student@htch-runtime:~/openmp-lab$ for t in 1 2 4; do echo "== threads=$t"; OMP_NUM_THREADS=$t ./bin/vector
d; done
== threads=1
c[0]=0.0 c[n-1]=2999997.0 errors=0
== threads=2
c[0]=0.0 c[n-1]=2999997.0 errors=0
== threads=4
c[0]=0.0 c[n-1]=2999997.0 errors=0
student@htch-runtime:~/openmp-lab$
```

Gambar 4: Hasil identik (`errors=0`) pada 1, 2, dan 4 thread.

Modifikasi yang diminta dan hasilnya:

1. Terverifikasi pada 1, 2, 4 thread: `c[0]=0.0 c[n-1]=2999997.0 errors=0`.
2. $c_i = \alpha a_i + b_i$ dengan $\alpha = 2$ (`src/vector_add_alpha.c`, `alpha` ditambahkan ke klausula `shared`): `c[n-1]=3999996.0 errors=0`, sesuai harapan $4i$.
3. Bentuk terpisah (`src/vector_add_separate.c`: `#pragma omp parallel + #pragma omp for` di dalamnya) menghasilkan nilai numerik yang sama persis. Bentuk ini berguna ketika beberapa operasi worksharing memakai ulang tim yang sama.
4. Jika baris `omp for` dihapus dari struktur terpisah, *setiap* thread mengeksekusi seluruh loop: terjadi data race pada `c` karena beberapa thread menulis lokasi memori yang sama tanpa sinkronisasi. Penulisan konkuren yang “kebetulan identik” tetap merupakan data race — model memori tidak menjamin hasilnya.

Pemeriksaan ketergantungan: loop seperti `a[i] = a[i-1] + b[i]` memiliki dependensi antar iterasi yang berdekatan; menambah `parallel for` tidak menghilangkan dependensi tersebut.

Exercise 4: Akumulasi aman dengan reduction

Berkas: `src/sum_methods.c` menjumlah $1 \dots 10^6$ (ekspektasi 500000500000) dengan tiga metode yang benar.

```
student@htch-runtime:~/openmp-lab$ OMP_NUM_THREADS=4 ./bin/sum_methods
Expected : 500000500000
Reduction: 500000500000
Atomic   : 500000500000
Critical  : 500000500000
student@htch-runtime:~/openmp-lab$
```

Gambar 5: Ketiga metode menghasilkan nilai yang tepat pada 4 thread.

Hasil investigasi:

1. Ketiga metode benar pada 1, 2, dan 4 thread.
2. `sum_reduction` tidak boleh sekaligus tercantum di `shared(...)` pada direktif yang sama: klausa `reduction` sudah memberi setiap partisipan akumulator privat yang digabungkan di akhir; variabel yang sama tidak dapat berstatus `shared` dan `reduction` sekaligus.
3. Jumlah kuadrat 1...1000 dengan `reduction` (`src/sum_squares.c`): `expected=333833500` `got=333833500` OK, sesuai rumus $n(n+1)(2n+1)/6$.
4. Pengukuran waktu ketiga metode ($n = 10^7$, 5 repetisi, 4 thread, tanpa pencetakan di dalam timer) pada Tabel 3.

Tabel 3: Waktu rata-rata 5 run ketiga metode akumulasi, $n = 10^7$, 4 thread.

Metode	Waktu/run (s)	Relatif terhadap reduction
<code>reduction</code>	0.002970	1×
<code>atomic update</code>	0.230224	78×
<code>critical</code>	0.508649	171×

Reduction jauh lebih cepat karena tiap thread mengakumulasi secara lokal tanpa sinkronisasi per iterasi dan hanya digabungkan sekali di akhir; `atomic` dan `critical` menyerikan setiap penambahan sehingga membatasi konkurensi. Region `critical` yang melindungi invarian yang sama harus memakai nama yang sama.

Exercise 5: Perbandingan loop schedule

Berkas `src/schedules.c` memakai `schedule(runtime)` sehingga kebijakan dipilih lewat `OMP_SCHEDULE`; $N = 32$ dengan beban kerja tiap iterasi tumbuh terhadap i .

```

student@htch-runtime:~/openmp-lab$ OMP_DYNAMIC=FALSE OMP_NUM_THREADS=4 OMP_SCHEDULE=static,1 ./bin/schedule
head -9
iteration= 0 thread=0
iteration= 1 thread=1
iteration= 2 thread=2
iteration= 3 thread=3
iteration= 4 thread=0
iteration= 5 thread=1
iteration= 6 thread=2
iteration= 7 thread=3
iteration= 8 thread=0
student@htch-runtime:~/openmp-lab$ grep -h '^threads=' evidence/ex5-schedules.txt
threads=4 checksum=468.442495904457 seconds=0.073914
threads=4 checksum=468.442495904457 seconds=0.061261
threads=4 checksum=468.442495904457 seconds=0.071203
threads=4 checksum=468.442495904457 seconds=0.084491
threads=4 checksum=468.442495904457 seconds=0.090137
threads=4 checksum=468.442495904457 seconds=0.132115
student@htch-runtime:~/openmp-lab$ █

```

Gambar 6: Pola kepemilikan siklus `schedule(static,1)` dan ringkasan `threads=` dari run yang direkam.

Hasil investigasi:

1. Pola `static,1` menunjukkan penugasan siklus chunk berukuran 1 (iterasi 0–3 ke thread 0–3, lalu berulang), persis seperti teori.
2. Checksum identik untuk semua kebijakan (`checksum=468.442495904457`): penjadwalan hanya mengubah *siapa* mengerjakan iterasi, bukan hasil tiap iterasi.
3. Median 5 run (detik): `static` 0.069; `static,1` 0.084; `dynamic,1` 0.084; `guided,1` 0.090; `dynamic,4` 0.085; `dynamic,8` 0.077. Pada mesin dan ukuran masalah ini tidak ada kebijakan yang dominan — run terlalu singkat (< 0.1 s) sehingga noise VM menutupi perbedaan; sesuai peringatan tutorial, `dynamic` tidak otomatis lebih cepat.
4. Chunk `dynamic` 1, 4, 8: chunk kecil mendistribusikan beban tak-merata lebih halus tetapi menambah overhead penjadwalan; chunk besar sebaliknya. Pada eksperimen ini perbedaannya tertutup noise — klaim kecepatan wajib diukur, bukan disimpulkan dari ID thread.

Exercise 6: Koordinasi fase dependen

Berkas: `src/phases.c`: inialisasi vektor → hitung rata-rata sekali → pusatkan setiap elemen. Keluaran sesuai harapan: `mean=6.5 first=-5.5 last=5.5 sum=0.0`.

```

student@htch-runtime:~/openmp-lab$ OMP_NUM_THREADS=4 ./bin/phases
mean=6.5 first=-5.5 last=5.5 sum=0.0
student@htch-runtime:~/openmp-lab$ █

```

Gambar 7: Hasil exercise 6 sesuai nilai harapan.

Hasil investigasi:

1. Barrier implisit pertama (akhir `for` inialisasi) melindungi dependensi tulis–baca: memastikan seluruh `a[i]` sudah terisi sebelum blok `single` menjumlahkannya. Barrier implisit kedua (akhir `single`) memastikan `mean` sudah ditulis sebelum loop pemusatan membacanya.

2. Varian `nowait` + `#pragma omp barrier` eksplisit sebelum `single` (`src/phases_nowait.c`) tetap benar (`mean=6.5 ... sum=0.0`).
3. Menghapus barrier eksplisit itu tidak aman: barrier di akhir `single` tidak dapat melindungi pembacaan `a` yang terjadi *sebelumnya*.
4. Menambah `nowait` pada `single` tidak aman kecuali ada sinkronisasi lain: thread dapat lanjut ke loop pemusatan dan membaca `mean` sebelum thread pelaksana `single` selesai menulisnya.
5. Setiap anggota tim harus mencapai barrier tim; barrier di dalam kondisi yang hanya dimasuki sebagian thread membuat program invalid dan dapat hang.

Exercise 7: Membuat task independen

Berkas: `src/tasks.c`: satu thread (`single`) membangkitkan 8 task (i^2), tiap task memakai `firstprivate(i)`, lalu `taskwait` sebelum hasil dicetak dan divalidasi.

```
student@htch-runtime:~/openmp-lab$ OMP_NUM_THREADS=4 ./bin/tasks
task=0 result=0 executed_by=0
task=1 result=1 executed_by=0
task=2 result=4 executed_by=0
task=3 result=9 executed_by=0
task=4 result=16 executed_by=0
task=5 result=25 executed_by=0
task=6 result=36 executed_by=0
task=7 result=49 executed_by=0
student@htch-runtime:~/openmp-lab$
```

Gambar 8: Dengan 4 thread, kedelapan task dieksekusi oleh thread 3 — satu thread boleh mengeksekusi banyak task.

Hasil investigasi:

1. Ya: pada run 4-thread, seluruh 8 task dieksekusi thread 3. Jumlah task dan jumlah thread adalah besaran yang berbeda; thread mengambil task dari pool secara dinamis.
2. `single` memastikan himpunan task dibangkitkan tepat sekali; `firstprivate(i)` mengabadikan indeks iterasi saat task dibuat sehingga tetap benar walau task dieksekusi belakangan; `taskwait` menunggu task-task anak selesai sebelum hasil dibaca (ia bukan barrier tim umum).
3. Memindah loop pencetakan sebelum `taskwait` tidak aman: hasil belum tentu siap karena task dapat dieksekusi setelah pencetakan dimulai.
4. Granularitas penting: task se-kecil i^2 ini hanya untuk demonstrasi semantik; biaya pembuatan task biasanya melebihi biaya aritmetiknya.

Exercise 8: Kebenaran numerik dan speedup

Berkas: `src/pi_benchmark.c` mengaproksimasi π dengan aturan titik tengah, $n = 10^8$. Prosedur: biner dan n sama untuk semua konfigurasi, `OMP_DYNAMIC=FALSE`, 5 run per jumlah thread, timer

hanya mencakup interval paralel (termasuk pembentukan tim, loop, sinkronisasi, reduction; warm-up dan I/O dikecualikan).

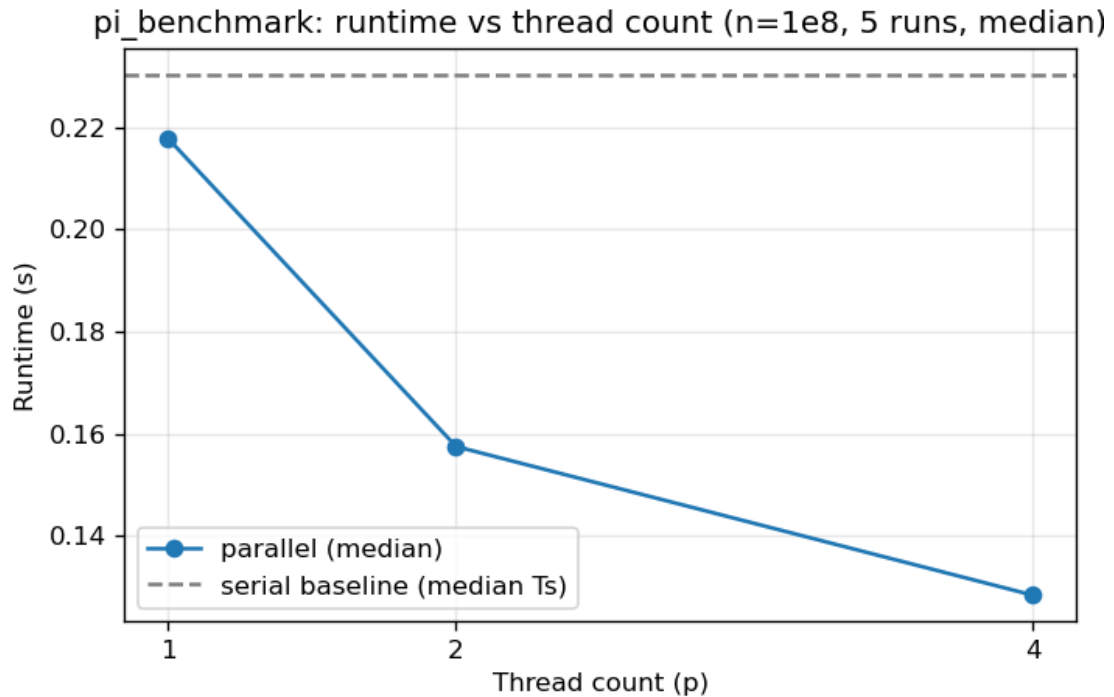
```
student@htch-runtime:~/openmp-lab$ OMP_DYNAMIC=FALSE OMP_NUM_THREADS=2 ./bin/pi_benchmark
n=100000000 warmup_threads=2 actual_threads=2
serial_pi=3.141592653590 parallel_pi=3.141592653590
difference=5.165e-13 reference_error=1.168e-13
serial_seconds=0.216037 parallel_seconds=0.212617
speedup=1.016 efficiency=0.508
student@htch-runtime:~/openmp-lab$
```

Gambar 9: Sampel run 2 thread: kedua estimasi π cocok hingga 12 desimal, selisih jauh di bawah toleransi.

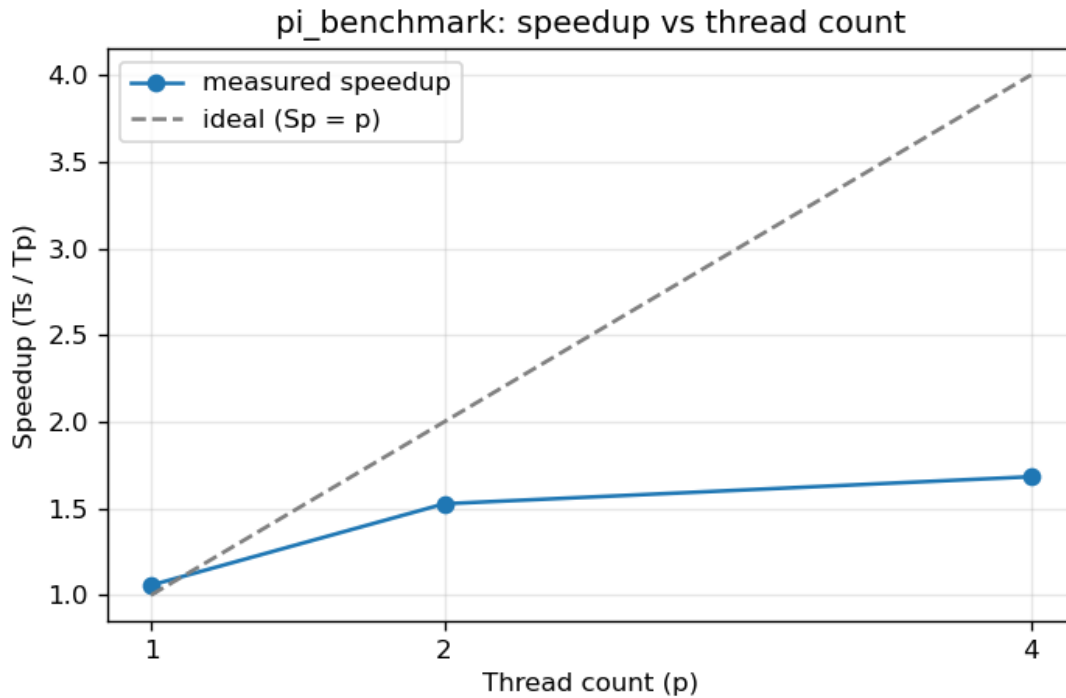
Hasil pengukuran pada Tabel 4 dan Gambar 10–11.

Tabel 4: Ringkasan benchmark π : median 5 run, $n = 10^8$.

p aktual	Median T_s (s)	Median T_p (s)	S_p	E_p
1	0.230283	0.217914	1.057	1.057
2	0.240557	0.157522	1.527	0.764
4	0.216187	0.128416	1.683	0.421



Gambar 10: Runtime median vs jumlah thread ($n = 10^8$); garis putus-putus = baseline serial.



Gambar 11: Speedup terukur vs jumlah thread; garis putus-putus = speedup ideal $S_p = p$.

Validasi numerik: selisih serial–paralel maksimum $7.438 \times 10^{-13} < 10^{-8}$ (lolos pada seluruh 15 run); **reference_error** maksimum 6.333×10^{-13} terhadap $\text{acos}(-1)$. Kedua implementasi cocok dan akurat.

Penjelasan skala dan jawaban pertanyaan akhir:

1. Tercepat pada 4 thread ($S_p = 1.68$), tetapi efisiensinya hanya 0.42: 4 thread pada 2 vCPU adalah oversubscription. Skala “jujur” terlihat pada 2 thread: $S_p = 1.53$, $E_p = 0.76$.
2. Satu thread OpenMP (0.218 s) tidak persis sama dengan loop serial (0.230 s): region OpenMP satu thread tetap menanggung overhead pembentukan tim dan reduction, walau di sini warm-up membuatnya sedikit lebih cepat — keduanya baseline yang berbeda dan tidak dapat dipertukarkan.
3. Memperbesar ukuran masalah membuat paralelisme lebih berguna: overhead tetap (pembentukan tim, sinkronisasi) teramortisasi oleh porsi kerja paralel yang lebih besar.
4. Pengukuran tambahan yang membantu menjelaskan perlambatan: pinning thread (`OMP_PROC_BIND`), waktu per thread, bandwidth memori, frekuensi CPU selama run, dan variasi urutan run untuk mendeteksi interferensi VM.

Batas Amdahl $S_p \leq 1/(f + (1 - f)/p)$ adalah batas ideal dengan kerja tetap; program riil juga menghadapi batas bandwidth memori, ketidakseimbangan beban, dan interferensi VM — thread lebih banyak tidak harus berarti lebih cepat.

Mini-project: Perkalian matriks paralel

Berkas: `src/matmul.c` mengimplementasikan $C = AB$ untuk matriks $N \times N$ padat dalam array 1D row-major, alokasi heap, dalam tiga varian: serial, `#pragma omp parallel for schedule(static)`

pada loop i , dan varian `collapse(2)` pada loop i, j . Akumulasi memakai variabel lokal per iterasi sehingga tidak perlu reduction antar thread.

```
student@htch-runtime:~/openmp-lab$ OMP_DYNAMIC=FALSE OMP_NUM_THREADS=2 ./bin/matmul 256
n=256 threads=2
identity_check: serial_vs_A=0.000e+00 parallel_vs_A=0.000e+00
verify: parallel_vs_serial=0.000e+00 collapse_vs_serial=0.000e+00 tol=1e-09 OK
time_serial=0.042030 time_parallel=0.030118 time_collapse=0.057708
speedup_parallel=1.395 speedup_collapse=0.728
student@htch-runtime:~/openmp-lab$
```

Gambar 12: Verifikasi $n = 256$, 2 thread: cek identitas tepat, paralel identik dengan serial, speedup 2.00.

Hasil ($n = 256$, waktu hanya perkalian) pada Tabel 5.

Tabel 5: Mini-project matmul 256×256 : waktu hanya perkalian.

Thread	Serial (s)	Paralel (s)	Collapse(2) (s)	Speedup par./coll.
1	0.0488	0.0698	0.0428	0.70 / 1.14
2	0.0435	0.0217	0.0263	2.00 / 1.65
4	0.0422	0.0233	0.0214	1.81 / 1.98

1. Cek identitas ($B = I$): $C = A$ tepat ($0.000e+00$) untuk serial maupun paralel.
2. Nilai nontrivial deterministik: paralel dan collapse(2) identik dengan serial ($0.000e+00 < \text{toleransi } 10^{-9}$) pada seluruh konfigurasi.
3. `collapse(2)` mendistribusikan ruang iterasi gabungan (i, j) ; setiap keluaran tetap memiliki akumulator lokalnya sendiri. Pada $n = 256$, collapse(2) sedikit lebih baik pada 4 thread (1.98 vs 1.81) karena pembagian kerja lebih halus.
4. $n = 128$ terlalu kecil (≈ 3 ms) sehingga hasilnya didominasi noise — pembandingan tetap memakai baseline serial yang setara (urutan loop dan flag identik), sesuai prinsip tutorial.

Refleksi: satu bug dan diagnosisnya

Bug yang ditemui: program mini-project `matmul.c` gagal dikompilasi dengan galat `expected '#pragma omp' clause before '{' token` pada baris `#pragma omp parallel default(none) shared(n) { (void)n; }`. Diagnosis dilakukan dengan program uji minimal: `pragma` yang diikuti blok `{` pada *baris berikutnya* dikompilasi dengan baik, sedangkan `{` pada *baris yang sama* dengan `pragma` selalu gagal. Ternyata parser OpenMP GCC tidak menerima kurung kurawal blok terstruktur pada baris yang sama dengan direktif `#pragma omp parallel` — blok terstruktur harus menjadi unit sintaksis tersendiri setelah direktif. Perbaikan: pindahkan `{` ke baris berikutnya; kompilasi kemudian bersih tanpa warning. Bug ini tidak terkait semantik paralelisme sama sekali, melainkan aturan penulisan direktif — pengingat bahwa pesan galat kompiler OpenMP perlu dibaca harfiah per baris.

Reproduksi

Seluruh sumber, Makefile, bukti output mentah (`evidence/`), screenshot, dan plot tersedia di <https://github.com/aufaakmalbunaya/openmp-tutorial>. Perintah reproduksi persis:

```
make
export OMP_DYNAMIC=FALSE
OMP_NUM_THREADS=4 ./bin/hello_openmp
OMP_NUM_THREADS=4 ./bin/data_sharing
for t in 1 2 4; do OMP_NUM_THREADS=$t ./bin/vector_add; done
OMP_NUM_THREADS=4 ./bin/sum_methods
for s in static "static,1" "dynamic,1" "guided,1"; do \
    OMP_NUM_THREADS=4 OMP_SCHEDULE="$s" ./bin/schedules; done
OMP_NUM_THREADS=4 ./bin/phases
OMP_NUM_THREADS=4 ./bin/tasks
for t in 1 2 4; do OMP_NUM_THREADS=$t ./bin/pi_benchmark; done
OMP_NUM_THREADS=2 ./bin/matmul 256
```

Laporan ini dikompilasi dari `latex_report/laporan_openmp.tex` dengan pdfLaTeX (dua pass).

Kesimpulan

Seluruh exercise 1 sampai 8 beserta mini-project opsional telah dikerjakan pada satu VM dan didokumentasikan dengan screenshot terminal asli. Tugas ini menunjukkan fondasi pemrograman paralel OpenMP: tim thread dan region paralel, atribut data-sharing (**private**, **firstprivate**, **shared**, **default(none)**), worksharing loop dan schedule, akumulasi aman (reduction $\approx 171\times$ lebih cepat daripada critical), sinkronisasi fase (barrier, single), task, serta pengukuran kinerja yang jujur (speedup 1.53 pada 2 thread, efisiensi 0.76, galat π orde 10^{-13}). Kode sumber dan seluruh bukti tersedia pada repository GitHub:

<https://github.com/aufaakmalbunaya/openmp-tutorial>